

cyclic

```
</activity>
<service
    android:name="com.mobilehackinglab.cyclicscanner.scanner.ScanService"
    android:exported="false"/>
```

analyzing the ScanService :



So the service once activated it will iterate over all files in external storage and run ScanEngine on it to see if it is infected or not

but the ScanEngine is running dangerous command on the external files :

```
public final boolean scanFile(File file) {
    Intrinsic.checkNotNullParameter(file, "file");
    try {
        String command = "toybox shalsum " + file.getAbsolutePath();
        Process process = new ProcessBuilder(new String[0]).command("sh", "-c", command).directory(Environment.getExternalStorageDirectory()).redirectErrorStream(true).start();
        InputStream inputStream = process.getInputStream();
        Intrinsic.checkNotNullExpressionValue(inputStream, "getInputStream(...)");
        Reader inputStreamReader = new InputStreamReader(inputStream, Charsets.UTF_8);
        BufferedReader bufferedReader = inputStreamReader instanceof BufferedReader ? (BufferedReader) inputStreamReader : new BufferedReader(inputStreamReader, 8192);
        try {
            BufferedReader reader = bufferedReader;

```

Within ScanEngine , the filename is concatenated directly with an OS command using file.getAbsolutePath() . This is a clear example of OS Command Injection vulnerability, as no sanitization is applied before the filename is used in the function that executes system commands.

If we follow the pattern filename; <os command> , we believe it is possible to execute arbitrary commands on the system.

file.txt;id;

